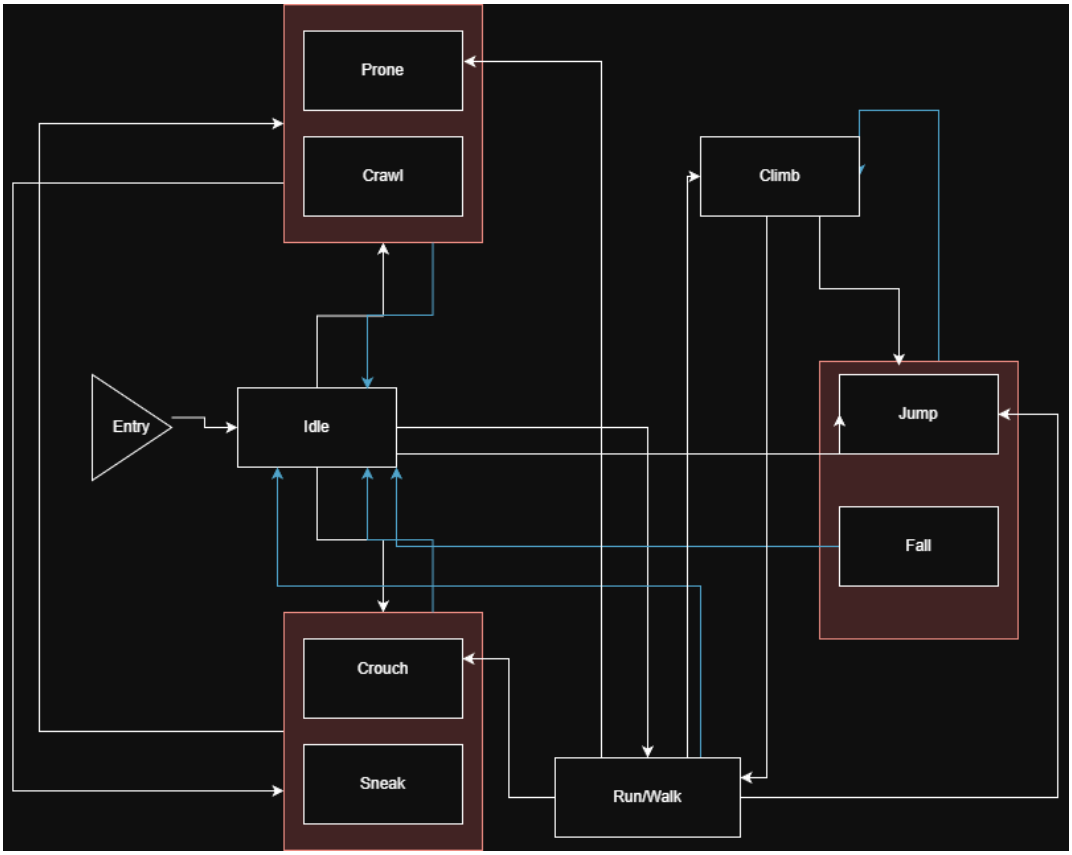
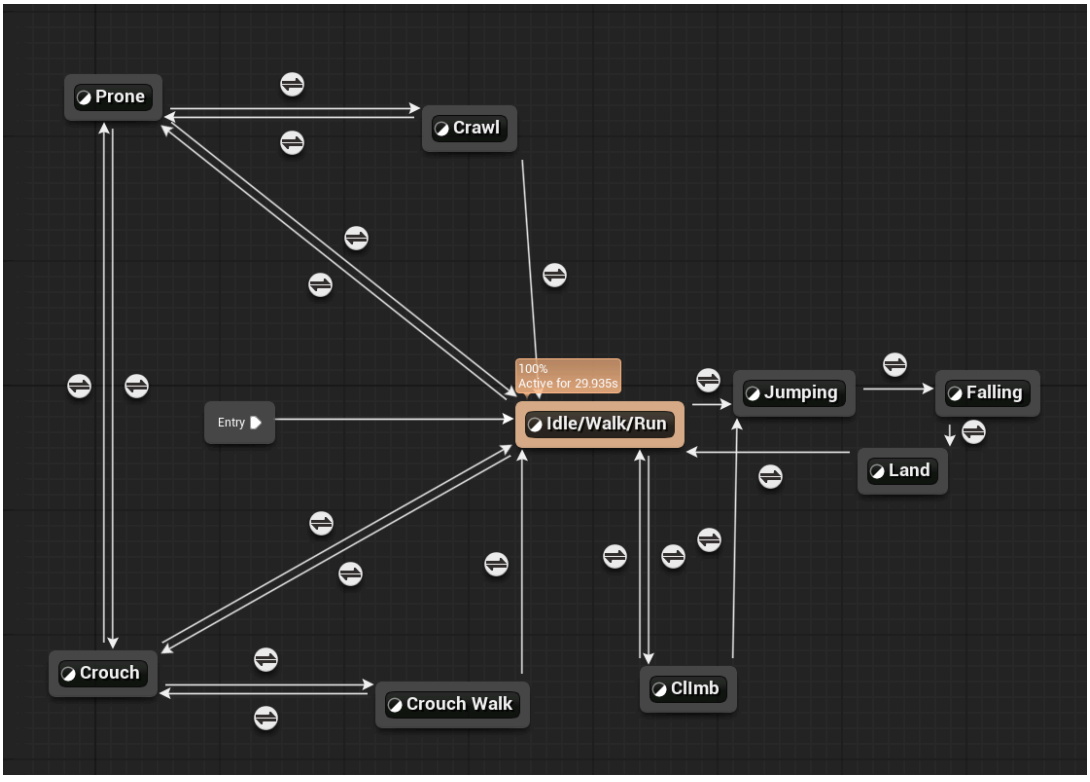
State Pattern Code Analysis

Repo: https://eae-git.eng.utah.edu/u1537288/playerstatepattern

Show a drawing of your state machine.





How did you implement the state pattern?

This project uses the State Pattern to control the player's movement behavior. Each movement state is implemented as its own class that inherits from a shared base state class. At any given time, only one state is active, and all player input is forwarded to that active state. The state itself decides what actions are allowed and whether a transition to another state should occur.

Instead of using large switch statements or chains of if/else logic, the system relies on polymorphism. The base state class defines a set of virtual functions, such as Enter(), Exit(), and input handlers like HandleJump() and HandleCrouch(). Each derived state overrides only the behaviors that make sense for that state. For example, the Prone state ignores jump input, the Climb state restricts movement to vertical motion only, and the Crouch state disables sprinting.

```cpp
class PlayerStateBase
{
public:
    virtual ~PlayerStateBase() = default;

    virtual void Enter(AMyCharacter* Character) {}
    virtual void Exit(AMyCharacter* Character) {}

    virtual void HandleMove(AMyCharacter* Character, bool bHasMoveInput) {}
    virtual void HandleJump(AMyCharacter* Character) {}
    virtual void HandleCrouch(AMyCharacter* Character) {}
    virtual void HandleProne(AMyCharacter* Character) {}
    virtual void HandleRun(AMyCharacter* Character, bool bPressed) {}
};
```

```cpp
void AMyCharacter::ChangeState(PlayerStateBase* NewState)
{
    if (CurrentState)
    {
        CurrentState->Exit(this);
    }

    CurrentState = NewState;

    if (CurrentState)
    {
        CurrentState->Enter(this);
    }
}
```

```cpp
class IdleState : public PlayerStateBase
{
public:
    void Enter(AMyCharacter* Character) override;
    void HandleMove(AMyCharacter* Character, bool bHasMoveInput) override;
    void HandleJump(AMyCharacter* Character) override;
    void HandleCrouch(AMyCharacter* Character) override;
    void HandleProne(AMyCharacter* Character) override;


ıss WalkRunState : public PlayerStateBase

›lic:
    void Enter(AMyCharacter* Character) override;
    void HandleMove(AMyCharacter* Character, bool bHasMoveInput) override;
    void HandleJump(AMyCharacter* Character) override;
    void HandleCrouch(AMyCharacter* Character) override;
    void HandleProne(AMyCharacter* Character) override;
    void HandleRun(AMyCharacter* Character, bool bPressed) override;


ıss JumpState : public PlayerStateBase
{
public:
    void Enter(AMyCharacter* Character) override;
};

class CrouchState : public PlayerStateBase
{
public:
    void Enter(AMyCharacter* Character) override;
    void Exit(AMyCharacter* Character) override;

    void HandleCrouch(AMyCharacter* Character) override;
    void HandleProne(AMyCharacter* Character) override;
};

class ProneState : public PlayerStateBase
{
public:
    void Enter(AMyCharacter* Character) override;
    void Exit(AMyCharacter* Character) override;

    void HandleCrouch(AMyCharacter* Character) override;
    void HandleProne(AMyCharacter* Character) override;
};

class ClimbState : public PlayerStateBase
{
public:
    void Enter(AMyCharacter* Character) override;
    void Exit(AMyCharacter* Character) override;

    void HandleJump(AMyCharacter* Character) override;
    void HandleCrouch(AMyCharacter* Character) override;
};
```

State transitions are handled through a ChangeState() function in the character class. When a transition occurs, the current state's Exit() function is called before switching to the new state and calling its Enter() function. This keeps transitions clean and centralized. The result is a clear and organized system where behavior depends entirely on the active state. This makes the movement logic easier to maintain, extend, and debug.

Did you also incorporate the Command pattern?

I did not need to use the Command pattern in this project, as there were already so many complexities in getting the state pattern to work for the first time. I just wanted to focus on that since that was the main focus. There was also no plan to make an AI that could copy the player, so another reason there was no need. If I did want to add it, then it would be for creating an AI that would chase the player, or race them in a race.

Catch All

If I were to do this assignment again, I would not do it the way I started, which was basically the way Unreal does it in the templates. I thought that getting the animations and states function that refactoring the code would be simple, but the State Pattern is something you start with and build off of. When I finally did implement it, it made adding the final, more complex state super straightforward. Unfortunately, I ran out of time to add more states, but I know with this pattern, it will be easy to build off of it.

Time Spent

I spent 14 hours on this assignment, partly due to the refactor, but also just learning how to create a character from scratch in Unreal, because I always used the template characters. I had to completely redo my animation state graph, but it made it so much easier to manage once that was refactored. I also tried modeling my character, but realized that it would take more time than I had, so it will be something I work on throughout the semester.